

Model View Controller in JavaScript

Struktur MVC

1. **Model** => enthält oder verwaltet die Daten
2. **View** => zeigt die Daten an
3. **Controller** => verbindet beides

Struktur in einer Applikation

```
/mvc-app  
  model.js  
  view.js  
  controller.js  
  app.js
```

Code Beispiele

model.js

```
let todos = [];  
  
export function addTodo(text) {  
  todos.push(text);  
}  
  
export function getTodos() {  
  return todos;  
}
```

View.js

```
export function renderTodos(todos) {  
  console.log("Todos:");  
  todos.forEach((t, i) => console.log(`${i + 1}. ${t}`));  
}
```

Controller.js

```
import { addTodo as add, getTodos } from './model.js';  
import { renderTodos } from './view.js';  
  
export function addTodo(text) {  
  add(text);  
  renderTodos(getTodos());  
}
```

Anwendung in der app.js

```
import { addTodo } from './controller.js';  
  
addTodo("Lernen");  
addTodo("Code schreiben");
```



MVC – Anwendungsbereiche

Grundidee

MVC wird eingesetzt, um **Code sauber zu strukturieren**, besonders wenn:

- Logik komplexer wird
- mehrere Entwickler zusammenarbeiten
- Anwendungen wachsen

1. Frontend (Vanilla JS & Frameworks)

Typische Nutzung

- Strukturierung von UI-Logik ohne Framework
- Grundlage vieler Frameworks

Beispiele

- Eigene JS Apps (Todo App, Dashboard)
- Architektur hinter Frameworks wie React (ähnlich, aber erweitert)

Wann sinnvoll?

- Wenn du **ohne Framework arbeitest**
- Wenn du **Logik und Darstellung trennen willst**

2. Backend (Node.js + Express)

Typische Nutzung

- Klassische Webserver-Struktur

Aufbau

- **Model** → Datenbank / Datenlogik
- **Controller** → Request-Handling
- **View** → JSON Response oder Templates

Beispiele

- REST APIs mit Express
- CRUD Anwendungen (User, Posts, etc.)

3. Große Anwendungen / Teams

Typische Nutzung

- Projekte mit mehreren Entwicklern

Vorteile

- Klare Verantwortlichkeiten
- Weniger Konflikte im Code
- Bessere Wartbarkeit

Beispiel

- Team A → arbeitet am Model
- Team B → baut Views
- Team C → entwickelt Controller
-

4. Skalierbare Anwendungen

Wann wird MVC wichtig?

Sobald deine App:

- viele Features bekommt
- wächst (10+ Dateien → 100+ Dateien)
- schwer wartbar wird

Typische Szenarien

- Admin Dashboards
- E-Commerce Systeme
- SaaS Anwendungen
-

5. Testbarkeit

Warum MVC hier stark ist

- Logik ist getrennt testbar



Beispiel

- Model → Unit testen
- Controller → API testen
- View → separat prüfen



MVC im Kontext von Framework (NEXTJS / REACTJS)

Model => Daten und Business Logic

- DB Zugriff (z. B. Prisma)
- Services / Utils

View => Rendering von JSX

- JSX / TSX Komponenten
- Server Components & Client Components

Controller => Datenlogik und Routing (lokale API)

- API Routes (/app/api/...)
- Server Actions
- Route Handler

Model

```
export async function getUsers() {  
  return db.user.findMany();  
}
```

Controller

```
export async function GET() {  
  return Response.json({ message: "Hello" });  
}
```

View

```
export default function Page() {  
  return <h1>Hello World</h1>;  
}
```